

Multi-Business Management Platform — Production Installation Guide

Platform: Windows 10/11 (production target) **Architecture:** Next.js app running as a Windows service via node-windows, with optional Electron desktop wrapper

Table of Contents

1. [Prerequisites](#)
 2. [Clone and Install](#)
 3. [Environment Configuration](#)
 4. [Database Setup](#)
 5. [Build](#)
 6. [SSL Certificates \(HTTPS\)](#)
 7. [Windows Service Installation](#)
 8. [QZ Tray Receipt Printing](#)
 9. [Electron Desktop App](#)
 10. [Client Machine Setup](#)
 11. [Deploying Updates](#)
 12. [Troubleshooting](#)
 13. [Service Management Reference](#)
-

1. Prerequisites

Install the following on the server machine before proceeding.

Node.js (via nvm for Windows)

Use [nvm-windows](#) to manage Node.js versions.

```
# After installing nvm-windows, install and use Node.js 20
nvm install 20
nvm use 20

# Verify
node --version    # v20.x.x
npm --version
```

PostgreSQL 18

Download and install from [postgresql.org](https://www.postgresql.org).

- Default port: 5432
- Note the password you set for the `postgres` user

- Ensure the service name is `postgresql-x64-18` (the app depends on this)

Verify:

```
"C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -c "SELECT version();"
```

Git

Download from git-scm.com. Git also installs `openssl` which is required for certificate generation.

Java (for QZ Tray printing)

Required only on machines that will print receipts via QZ Tray.

Download from adoptium.net (Java 11+ LTS recommended).

```
java -version # verify
```

2. Clone and Install

```
git clone <your-repository-url> multi-business-multi-apps
cd multi-business-multi-apps

npm install
```

Note: `npm install` triggers a `postinstall` script. This is expected and safe.

3. Environment Configuration

Create `.env.local` in the project root (this file is never committed to git):

```
# — Database —————
DATABASE_URL="postgresql://postgres:YOUR_PASSWORD@localhost:5432/multi_business_db"

# — Auth —————
# Generate a secret: node -e
# "console.log(require('crypto').randomBytes(32).toString('base64'))"
NEXTAUTH_SECRET="your-generated-secret-here"
NEXTAUTH_URL="http://localhost:8080" # update to https:// after SSL is set up

# — App —————
PORT=8080
NODE_ENV=production
```

```
APP_NAME="Multi-Business Management Platform"
ADMIN_EMAIL="your-admin@email.com"
ENCRYPTION_KEY="generate-a-64-char-hex-string"

# — Sync Service —————
SYNC_NODE_ID="generate-a-16-char-hex-string"
SYNC_NODE_NAME="sync-node-MACHINENAME"
SYNC_SERVICE_PORT=8765
SYNC_HTTP_PORT=8080
SYNC_AUTO_START=true
SYNC_LOG_LEVEL=warn
SYNC_REGISTRATION_KEY="generate-a-64-char-hex-string"

# — QZ Tray Printing (add after running npm run qz:generate-cert) —————
# QZ_PRIVATE_KEY=<base64 value from generate-qz-cert output>
# QZ_CERTIFICATE=<base64 value from generate-qz-cert output>
```

Generating secret values:

```
# NEXTAUTH_SECRET (32 random bytes, base64)
node -e "console.log(require('crypto').randomBytes(32).toString('base64'))"

# ENCRYPTION_KEY (32 random bytes, hex)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"

# SYNC_NODE_ID (8 random bytes, hex)
node -e "console.log(require('crypto').randomBytes(8).toString('hex'))"

# SYNC_REGISTRATION_KEY (32 random bytes, hex)
node -e "console.log(require('crypto').randomBytes(32).toString('hex'))"
```

4. Database Setup

Create the database

```
"C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -c "CREATE DATABASE
multi_business_db;"
```

Run migrations

```
npm run db:deploy
```

Critical: Never use `prisma db push` on production. Always use `db:deploy` which runs `prisma migrate deploy`.

Verify

```
npx prisma migrate status --env-file .env.local
```

Expected: **Database schema is up to date!**

5. Build

```
npm run build
```

This runs:

1. **prisma generate** — generates the Prisma client
2. **next build** — compiles the Next.js app into **.next/**
3. **tsc --project tsconfig.server.json** — compiles **server.ts** into **dist/server.js**

Important: **dist/server.js** must exist for the service to start. If it's missing after a **git pull**, run **npm run build:server** (faster than a full rebuild).

6. SSL Certificates (HTTPS)

The app auto-detects HTTPS by checking the **certs/** folder for **.pem** files. If certs are present, the server starts on HTTPS; otherwise it falls back to HTTP.

One shared certificate covers every server on the LAN — generated once on a single "certificate authority" machine and copied out to each server. Only that one machine needs **mkcert** installed; every other server just receives the already-generated files.

6a. One-time setup — the certificate authority machine only

Pick one machine to be the permanent cert authority (this only needs to be done once, ever — subsequent servers never repeat this step). **mkcert** isn't on the Node/Git installers used elsewhere in this guide, so download it directly rather than assuming Scoop/Chocolatey are available:

1. Download the Windows binary from the [mkcert releases page](#) — grab **mkcert-vX.Y.Z-windows-amd64.exe**.
2. Rename it to **mkcert.exe** and place it somewhere on **PATH** (e.g. **C:\mkcert\mkcert.exe**, then add **C:\mkcert** to your **PATH** environment variable) — no installer, no admin rights needed for the binary itself.
3. Install the local CA once:

```
mkcert -install
```

4. In this repo, maintain the list of every server's IP in `scripts/lan-server-ips.json`, then generate the shared certificate:

```
npm run cert:generate
```

This writes `certs/<ip>+N.pem` and `certs/<ip>+N-key.pem`, covering every IP in that list plus `localhost/127.0.0.1`.

`openssl` (used only to verify a cert's coverage, in `install-server-cert.ps1`) is **not** a separate install — it ships with Git for Windows, already a prerequisite for every server per §1. No server other than the cert-authority machine ever needs `mkcert` itself.

6b. Adding or setting up a new server (do this on every other server)

1. On the cert-authority machine, add the new server's IP and regenerate in one step:

```
npm run cert:generate -- --add <new-server-ip>
```

(Existing servers are unaffected — they keep using their already-installed cert with no changes.)

2. Copy the two regenerated files (`certs/<ip>+N.pem` and `certs/<ip>+N-key.pem`) to the new server — USB stick, shared folder, SCP; the `certs/` folder is in `.gitignore`, so this is always a manual/out-of-band transfer, never a `git pull`.
3. On the new server, from the app root:

```
npm run cert:install -- "<path containing the copied files>"
```

This places the files in `certs/`, warns if the server's own IP wasn't actually included in the certificate (the most common mistake — forgetting step 1), and restarts the service for you.

4. Confirm the service log shows `[Server] HTTPS enabled – certs loaded from ./certs/`.

Every machine that opens the app in a browser also needs to trust the root CA once — that's covered separately in §10, since it's a one-time step per *client*, not per server.

6c. Certificate renewal

Check the current expiry:

```
openssl x509 -noout -enddate -in certs/<name>+N.pem
```

Renewing is the same as adding a server, minus changing the IP list:

1. On the cert-authority machine: `npm run cert:generate`

2. Redistribute the regenerated `.pem` files to **every** server this time (a renewal changes the file every existing server uses, unlike adding one new server)
3. Restart every server
4. No client reinstallation needed — the root CA outlives any individual leaf certificate

Update NEXTAUTH_URL

After SSL is working, update `.env.local` on that server to use its own IP:

```
NEXTAUTH_URL="https://<this-server-ip>:8080"
```

Restart the service after this change.

7. Windows Service Installation

The app runs as a Windows service via `node-windows`. The service starts PostgreSQL → runs migrations → seeds reference data (including the default `admin@business.local` / `admin123` login — no separate step needed) → starts the Next.js app.

Install (run as Administrator)

```
npm run service:install
```

This registers the `MultiBusinessSyncService` in Windows Services and creates the daemon files in `windows-service/daemon/`.

The service depends on `postgresql-x64-18`. PostgreSQL must be installed and its service name must match.

Start

```
npm run service:start
```

Verify it's running

```
npm run service:status  
# or  
sc query "MultiBusinessSyncService"
```

View logs

```
# Live tail of application output
Get-Content "windows-service\daemon\multibusinesssyncservice.out.log" -Wait -Tail
30

# Live tail of errors
Get-Content "windows-service\daemon\multibusinesssyncservice.err.log" -Wait -Tail
30
```

Healthy startup sequence ends with:

```
[Server] HTTPS enabled – cert: <filename>.pem, key: <filename>-key.pem
[Server] listening on https://localhost:8080
```

8. QZ Tray Receipt Printing

QZ Tray is a Java desktop app that lets the browser print to local/network printers without browser print dialogs.

Install QZ Tray

Download version **2.2.6** from qz.io and install on each machine that will print receipts. Start QZ Tray — it runs in the system tray.

First connection (before signing is set up)

Without the signed-certificate setup below, QZ Tray shows an **Allow/Deny** popup the first time the app tries to print on each machine:

1. Click **Allow**
2. Check "**Remember this decision**"
3. Done — future connections on that machine are automatic

This is fine for a small number of machines; the signed-certificate setup below eliminates the popup entirely, which matters more as client machines scale up.

Set up signed certificates (eliminates the Allow/Deny popup)

How this differs from SSL certs: The SSL `rootCA.pem` must be a physical file on the server. The QZ certificate is different — the server reads it from `.env.local` env vars, not from a file. Only client machines (where QZ Tray is installed) need the physical `qz-certificate.pem` file.

Step 1 — Generate the signing certificate (run once on the server or any dev machine):

```
npm run qz:generate-cert
```

This outputs two environment variable values **and** creates `certs/qz-certificate.pem`.

Step 2 — Add env vars to the server's `.env.local`:

```
QZ_PRIVATE_KEY=<value from script output>
QZ_CERTIFICATE=<value from script output>
```

The server serves the certificate to the browser via `/api/qz/certificate` using these env vars. **No file copy to the server is needed** — the env vars are sufficient.

Step 3 — Distribute `qz-certificate.pem` to each client machine running QZ Tray:

Copy `certs/qz-certificate.pem` to each machine where QZ Tray is installed (USB, shared folder, etc.), then:

1. Right-click QZ Tray icon in system tray
2. **Advanced** → **Site Manager**
3. Click **Add Certificate**
4. Select the `qz-certificate.pem` file
5. Restart QZ Tray

Step 4 — Rebuild and restart the service:

```
npm run build
npm run service:restart
```

From this point, QZ Tray will silently trust all print requests from the app — no more popups.

File	Server needs it?	QZ Tray machine needs it?
<code>rootCA.pem</code>	✓ Physical file in <code>certs/</code>	✓ Run <code>setup-ssl.bat</code>
SSL cert + key <code>.pem</code>	✓ Physical file in <code>certs/</code>	✗ Not needed
<code>qz-certificate.pem</code>	✗ Env var in <code>.env.local</code> only	✓ Add to QZ Tray trusted list

9. Electron Desktop App

Electron wraps the web app as a desktop application and launches automatically at Windows startup.

Install the startup shortcut

```
npm run electron:install-startup
```

This creates a shortcut in `%APPDATA%\Microsoft\Windows\Start Menu\Programs\Startup` that launches Electron when the user logs in.

Run this command **after** setting up SSL certificates. The shortcut is regenerated with the correct `http://` or `https://` protocol based on what certs are present.

Launch manually

```
npm run electron:start
```

How Electron detects HTTPS

`electron/main.js` auto-detects certs in the `certs/` folder and loads `https://localhost:8080` if they exist, otherwise `http://localhost:8080`. It also has a certificate-error handler to trust the self-signed mkcert cert.

10. Client Machine Setup

Every Windows machine that opens the app in Chrome or Edge needs to trust the SSL certificate once.

Option A — One-click (recommended)

1. Copy `certs/setup-ssl.bat` and `certs/rootCA.pem` to the client machine
2. Double-click `setup-ssl.bat`
3. Restart Chrome/Edge

Option B — Manual

1. Copy `certs/rootCA.pem` to the client machine
2. Press `Win+R` → type `certmgr.msc` → Enter
3. Expand **Trusted Root Certification Authorities** → right-click **Certificates**
4. **All Tasks** → **Import** → select `rootCA.pem`
5. Restart Chrome/Edge

QZ Tray on client machines

Each client machine that prints needs:

1. Java 11+ installed
2. QZ Tray 2.2.6 installed and running
3. `certs/qz-certificate.pem` added to QZ Tray (see Section 8 Step 2)

11. Deploying Updates

Stop the service first. `npm install` (its `postinstall` hook runs `prisma generate`) and `npm run build` (which also runs `prisma generate`) both need to overwrite the Prisma query engine DLL — and Windows refuses to do that while any running process, including this app's own service, still has it loaded. Building before stopping the service fails with an `EPERM`/file-lock error on that DLL.

```
# 1. Stop the service – releases the Prisma engine lock
npm run service:stop

# 2. Pull latest code
git pull

# 3. Install any new dependencies
npm install

# 4. Run new migrations
npm run db:deploy

# 5. Rebuild
npm run build

# 6. Start the service back up
npm run service:start
```

If only `server.ts` changed (no UI changes), you can skip the full build and just run `npm run build:server` instead of step 5.

You can skip step 5 entirely if you'd rather let the service rebuild itself: the hybrid service wrapper (\$7) detects a stale commit on startup and runs `npm run build` on its own before serving requests — since the service is already stopped at that point (step 1), it hits none of the locking issues a manual mid-run build would. Either way, the service must be stopped before step 3.

After a failed migration

If a migration fails and leaves the database in a `failed` state:

```
# 1. Mark the failed migration as rolled back
npx prisma migrate resolve --rolled-back "MIGRATION_NAME" --env-file .env.local

# 2. Re-apply (with the fixed migration SQL)
npm run db:deploy
```

12. Troubleshooting

Service starts then stops immediately

Check the error log:

```
Get-Content "windows-service\daemon\multibusinesssyncservice.err.log" -Tail 50
```

Common causes:

- **dist/server.js not found** — run `npm run build:server`
- **PostgreSQL not running** — start PostgreSQL service first
- **Port 8080 in use** — find and stop the conflicting process
- **Migration failed** — see "After a failed migration" above

Migration lock stuck

Symptom: **Migration lock present, waiting 5s...** repeating in logs.

```
npm run service:stop
npm run db:clear-locks    # or manually delete .migration.lock file
npm run service:start
```

If no lock file but still stuck, reboot the server — a zombie process may be holding the lock.

App running on HTTP instead of HTTPS

The server logs this on startup:

```
[Server] HTTP only – no certs found in ./certs/
```

Fix:

1. Ensure **certs/** folder exists in the app root (`C:\Users\...\multi-business-multi-apps\certs\`)
2. Ensure it contains both a **.pem** cert file and a **-key.pem** key file
3. Ensure this server's own IP is actually in **scripts/lan-server-ips.json** and was included when the certificate was generated — a cert loading fine but still showing "not secure"/mismatch warnings usually means this, not a missing-file problem. Re-run `npm run cert:install` — it warns explicitly if the local IP isn't covered.
4. Ensure **dist/server.js** was compiled from the latest **server.ts** (run `npm run build:server`)
5. Restart the service

QZ Tray popup appears every print job

The **QZ_PRIVATE_KEY** and **QZ_CERTIFICATE** are not set in **.env.local**, or the cert hasn't been added to QZ Tray. Follow Section 8 fully.

QZ Tray popup not appearing at all

Make sure the app is actually being accessed over HTTPS (QZ Tray's popup is triggered by the page, and won't fire from a plain **http://** origin), then refresh the printers page.

QZ Tray shows "connected" but can't list printers

Click **Allow** on the QZ Tray popup and check **"Remember this decision"** — a dismissed or denied popup leaves QZ Tray connected at the socket level but unauthorized to enumerate printers.

Chrome/Edge shows "Not secure" despite certs being installed

The root CA hasn't been trusted on *that specific client machine* yet — this is a per-client step, unrelated to whether the server's cert is correct. Run `certs\setup-ssl.bat` on that machine (see Section 10) and restart the browser.

Electron shows ERR_TIMED_OUT

The server hasn't finished starting when Electron tried to connect. Wait 30–60 seconds for the full startup sequence to complete, then re-open Electron. For a permanent fix, the startup shortcut waits before launching — re-run `npm run electron:install-startup`.

NEXTAUTH JWT session errors after rebuilding

Old session cookies become invalid when `NEXTAUTH_SECRET` changes or the server URL changes from `http://` to `https://`. Users just need to log out and log back in. No code fix required.

13. Service Management Reference

Command	Description
<code>npm run service:install</code>	Install as Windows service (run as Admin)
<code>npm run service:uninstall</code>	Remove Windows service (run as Admin)
<code>npm run service:start</code>	Start the service
<code>npm run service:stop</code>	Stop the service
<code>npm run service:restart</code>	Stop then start
<code>npm run service:status</code>	Show service status
<code>npm run db:deploy</code>	Run pending database migrations
<code>npm run db:clear-locks</code>	Clear stuck migration locks
<code>npm run build</code>	Full build (Next.js + server.ts)
<code>npm run build:server</code>	Compile server.ts only (fast)
<code>npm run electron:install-startup</code>	Install/update Electron startup shortcut
<code>npm run qz:generate-cert</code>	Generate QZ Tray signing certificate
<code>npm run cert:generate</code>	Regenerate the shared LAN HTTPS certificate (cert-authority machine only)
<code>npm run cert:generate -- --add <ip></code>	Add a new server's IP and regenerate in one step
<code>npm run cert:install -- <path></code>	Install a copied certificate on this server and restart

Log file locations

Log	Path
Application stdout	windows-service\daemon\multibusinesssyncservice.out.log
Application stderr	windows-service\daemon\multibusinesssyncservice.err.log
Wrapper log	windows-service\daemon\multibusinesssyncservice.wrapper.log

Quick Reference — Fresh Server Setup

```
# 1. Install Node.js (via nvm), PostgreSQL 18, Git

# 2. Clone and install
git clone <repo> multi-business-multi-apps
cd multi-business-multi-apps
npm install

# 3. Create .env.local (fill in your values)
#   See Section 3 for all required variables

# 4. Create database
"C:\Program Files\PostgreSQL\18\bin\psql.exe" -U postgres -c "CREATE DATABASE
multi_business_db;"

# 5. Run migrations
npm run db:deploy

# 6. Build
npm run build

# 7. Get the shared HTTPS certificate onto this server (see Section 6)
#   On the cert-authority machine: npm run cert:generate -- --add <this-server-
ip>
#   Copy the two regenerated certs/*.pem files here, then:
npm run cert:install -- "<path to the copied files>"

# 8. Install Windows service (Admin shell)
npm run service:install
npm run service:start

# 9. Set up QZ Tray signing (eliminates print popups)
npm run qz:generate-cert
# → add QZ_PRIVATE_KEY and QZ_CERTIFICATE to .env.local
# → add certs/qz-certificate.pem to QZ Tray trusted list
npm run service:restart

# 10. Install Electron startup shortcut
npm run electron:install-startup

# 11. Trust SSL cert on each client machine
```

```
# Copy certs/setup-ssl.bat + certs/rootCA.pem → run setup-ssl.bat on each machine
```

Default admin login: `admin@business.local` / `admin123` — **change immediately after first login.**